

Linear pipeline

Without pipelining a processor (non-pipeline processor) fetches one instruction, completes its execution, then fetches the next instruction. So while fetching the next instruction the arithmetic part of the processor remains idle, it must wait until it gets the next instruction.

Pipeline consists of a few segments and executes instructions in an overlapping fashion to reduce the overall execution time

Example: The following is a four segment pipeline. The segments are S1, S2, S3, S4. The segment S1 receives input and the output is obtained from S4 (shown using arrow). Let us consider that this four segment pipeline is an instruction pipeline. S1 is the fetch segment, S2 is the decode segment, S3 is the fetch operand segment, S4 is the execute and store result segment. Suppose you have a program having four instructions (I1, I2, I3, I4) and these instructions are independent on each other.



	CC1	CC2	CC3	CC4	CC5	CC6	CC7
S1	I1	I2	I3	I4			
S2		I1	I2	I3	I4		
S3			I1	I2	I3	I4	
S4				I1	I2	I3	I4

Details of execution of the 4 instructions using the above pipeline:

In CC1 I1 enters into the segment S1 of the pipeline for fetch operation.

In CC2 I1 enters into the segment S2 of the pipeline for decode operation whereas I2 enters into the segment S1 of the pipeline for fetch operation.

In CC3 I3 enters into the segment S1 for fetch, I2 enters into the segment S2 for decode, I1 enters into the segment S3 for fetch operand.

In CC4 I4 enters into the segment S1 for fetch, I3 enters into the segment S2 for decode, I2 enters into the segment S3 for fetch operand, I1 enters into the segment S4 for execution and store. So the execution of I1 is over in CC4.

In CC5 I2 enters into the segment S4 i.e. the execution of I2 is over in CC5.

In CC6 the execution of I3 is over and in CC7 the execution of I4 is over.

Therefore total 7 clock cycles are required to complete the execution of 4 instructions (I1, I2, I3, I4) using a 4 segment pipeline (S1, S2, S3, S4).

In CC4 all the four segments of the pipeline are busy and so the pipeline is full in CC4.

Linear pipeline processor

Cascade of processor segments which are linearly connected to perform a fixed function over a stream of data flowing from one end to the other

Two linear pipeline model

Asynchronous model (Fig 1 is a 3 segment asynchronous linear pipeline)

Synchronous model (Fig 2 is a 3 segment synchronous linear pipeline)

Asynchronous model

Data flow between the adjacent segments is controlled by a handshaking protocol

Segment S_i (i^{th} segment, $1 \leq i \leq 3$) sends a ready signal (Ready in Fig 1) to segment S_{i+1} when ready to transmit

S_{i+1} returns an acknowledgement signal (Ack in Fig 1) after receiving data (Data in Fig 1) from S_i

May have a variable throughput rate as different segments of the pipeline has different amount of delay

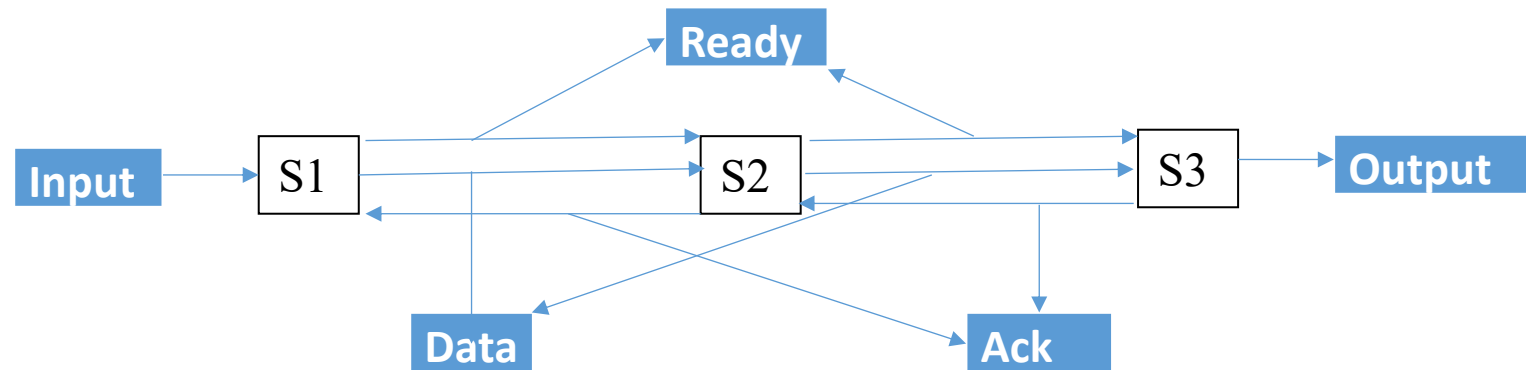


Fig.1

Synchronous model

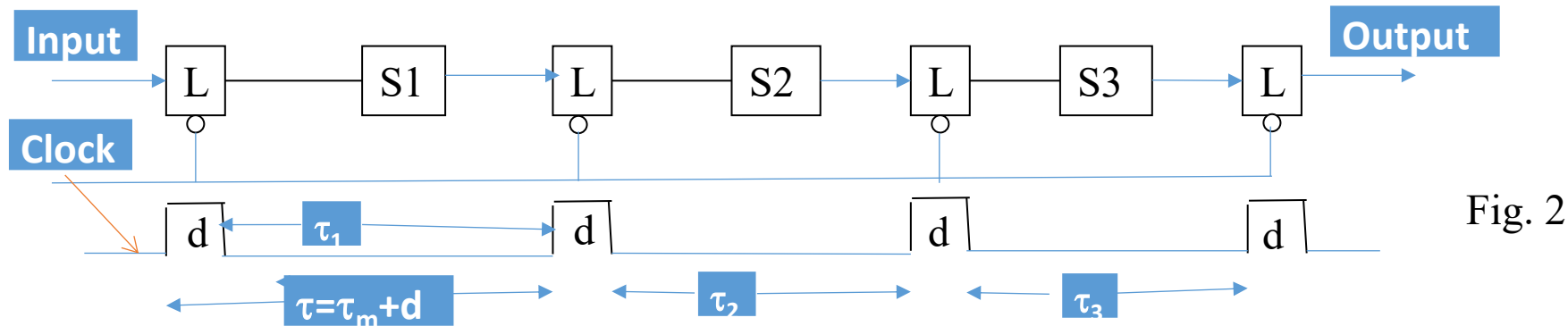
Uses clocked latches (L in Fig 2) to interface between segments

Latches are made with master/slave flip flops to isolate input from output

[Master is working and receiving input, slave is not working; Slave is working and giving output, master is not working; So input is isolated from output;]

All the latches transfer data to the next segment simultaneously at the time of arrival of each clock pulse

Pipeline segments are combinational logic circuits



Clocking and timing signal (shown in Fig 2)

τ = Clock cycle time

τ_i = Time delay of segment S_i

d = Latch delay

τ_m = Maximum segment delay = $\max(\tau_1, \tau_2, \tau_3)$

$\tau = \tau_m + d$, τ_m is very much greater than d

$f = 1/\tau$ = Clock frequency

Example: A pipeline has 4-segment, S_1, S_2, S_3, S_4 . S_1 takes 2 ns, S_2 takes 4 ns, S_3 takes 3 ns, S_4 takes 5 ns to complete their task. So $\tau_m = \max(2, 4, 3, 5) = 5$ ns

Reservation table: It specifies the utilization pattern of successive segments in a pipeline. The reservation table of a 4 segment pipeline is shown below. Four clock pulses (CC1 to CC4) are needed to complete the execution of a single instruction using this pipeline. The cross mark (X) at the intersection of i^{th} row ($1 \leq i \leq 4$) and j^{th} column ($1 \leq j \leq 4$) indicates the utilization of i^{th} segment (S_i) in j^{th} clock cycle (CC $_j$).

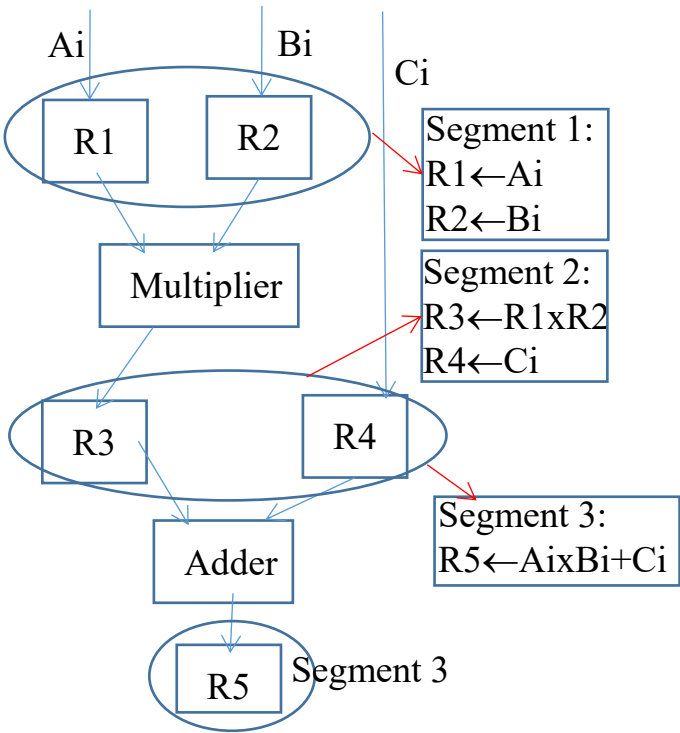
	CC1	CC2	CC3	CC4
S1	X			
S2		X		
S3			X	
S4				X

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8
S1	I1	I2	I3	I4	I5			
S2		I1	I2	I3	I4	I5		
S3			I1	I2	I3	I4	I5	
S4				I1	I2	I3	I4	I5

Space time diagram: It is the utilization of spaces in a pipeline (segments in a pipeline) in time (clock cycles). The space time diagram for the execution of 5 independent instructions (I1, I2, I3, I4, I5) using 4 segment pipeline (S1, S2, S3, S4) is shown below.

So 8 (=4+(5-1)) clock cycles are required to complete the execution of 5 instructions using a 4 segment pipeline. Moreover, in CC4 and CC5 all the 4 segments of the pipeline are busy, the pipeline is full and all the segments are executing instructions in overlapping fashion.

Arithmetic pipeline: Compute $A_i \cdot B_i + C_i$ for $i = 1$ to 7



Content of the registers at different clock cycle						Computation of first task (i=1) is over in 3 rd clock cycle Computation of 2 nd (i=2), 3 rd (i=3), 4 th (i=4), 5 th (i=5), 6 th (i=6), 7 th (i=7) are over in 4 th , 5 th , 6 th , 7 th , 8 th , 9 th clock cycle respectively So total 9 clock cycles are required to complete the computation using pipeline
Clock cycle	R1	R2	R3	R4	R5	
1	A1	B1	-	-	-	
2	A2	B2	$A1 \cdot B1$	C1	-	
3	A3	B3	$A2 \cdot B2$	C2	$A1 \cdot B1 + C1$	
4	A4	B4	$A3 \cdot B3$	C3	$A2 \cdot B2 + C2$	
5	A5	B5	$A4 \cdot B4$	C4	$A3 \cdot B3 + C3$	
6	A6	B6	$A5 \cdot B5$	C5	$A4 \cdot B4 + C4$	
7	A7	B7	$A6 \cdot B6$	C6	$A5 \cdot B5 + C5$	
8	-	-	$A7 \cdot B7$	C7	$A6 \cdot B6 + C6$	
9	-	-	-	-	$A7 \cdot B7 + C7$	

Computation needs $7 \cdot 3$ (number of tasks * number of segments) clock cycles without pipeline. Hence speedup of the pipeline is $(7 \cdot 3) / (3 + (7 - 1))$

Instruction pipeline: Basic DLX pipeline. DLX is pronounced as Deluxe. It is a simple load store architecture



Key Stages of the DLX pipeline

The standard DLX pipeline consists of five stages:

IF (Instruction Fetch): The instruction is fetched from memory, and the Program Counter (PC) is incremented.

ID (Instruction Decode/Register Fetch): The instruction is decoded, and operands are read from the register file.

EX (Execute/Effective Address): The ALU performs operations (arithmetic/logic) or calculates memory addresses.

MEM (Memory Access): The processor accesses data memory (load/store) or completes branch instructions.

WB (Write Back): The results of arithmetic, logic, or load instructions are written back to the register file.

Pipelining speedup is the ratio of execution time without pipelining to time with pipelining, measuring performance enhancement.

Superscalar processor

Uses multiple independent instruction pipeline, each pipeline consists of multiple segments
Fetches multiple instructions at a time, then attempts to find nearby instructions that are independent on one another for parallel execution using multiple pipeline

Example: In the following example the superscalar processor has two independent instruction pipelines (Pipeline 1, Pipeline 2). Each pipeline has 4 segments (S1, S2, S3, S4). Such a processor has degree 2 as it can issue two independent instructions in each clock cycle i.e. 2 independent instructions can enter into 2 pipelines (Pipeline 1, Pipeline 2) in each clock cycle.

CC1	CC2	CC3	CC4	CC5	CC6
S1(I1)	S2 (I1)	S3 (I1)	S4 (I1)	Pipeline 1 Pipeline 2	
S1(I2)	S2 (I2)	S3 (I2)	S4 (I2)		
	S1 (I3)	S2 (I3)	S3 (I3)	S4 (I3)	
	S1 (I4)	S2 (I4)	S3 (I4)	S4 (I4)	
		S1 (I5)	S2 (I5)	S3 (I5)	S4 (I5)
		S1 (I6)	S2 (I6)	S3 (I6)	S4 (I6)

Example of dependent and independent instruction:
I1: ADD R1, R2
I2: SUB R1, R3
Here the instruction I1 modifies R1 during addition operation and the instruction I2 uses the modified R1 during subtraction operation. So I1 and I2 are dependent instructions. The execution of I2 can be started only after the completion of execution of I1. So the above superscalar processor can't issue I1 and I2 in the same clock cycle.
But if I2 is SUB R3, R4 then I2 becomes independent on I1 and can be executed simultaneously with I1 using 2 independent pipelines (Pipeline 1, Pipeline 2) in the above superscalar processor.

Now consider a superscalar processor of degree m ($m > 1$) that is the processor has m number of independent pipelines (Pipeline 1, Pipeline 2,.....Pipeline m). Each pipeline has K number of segments. N is the total number of instructions. So $K + \frac{N-m}{m}$ is the required number of clock cycles to complete the execution of N number of instructions by this processor.

Explanation of $K + \frac{N-m}{m}$:

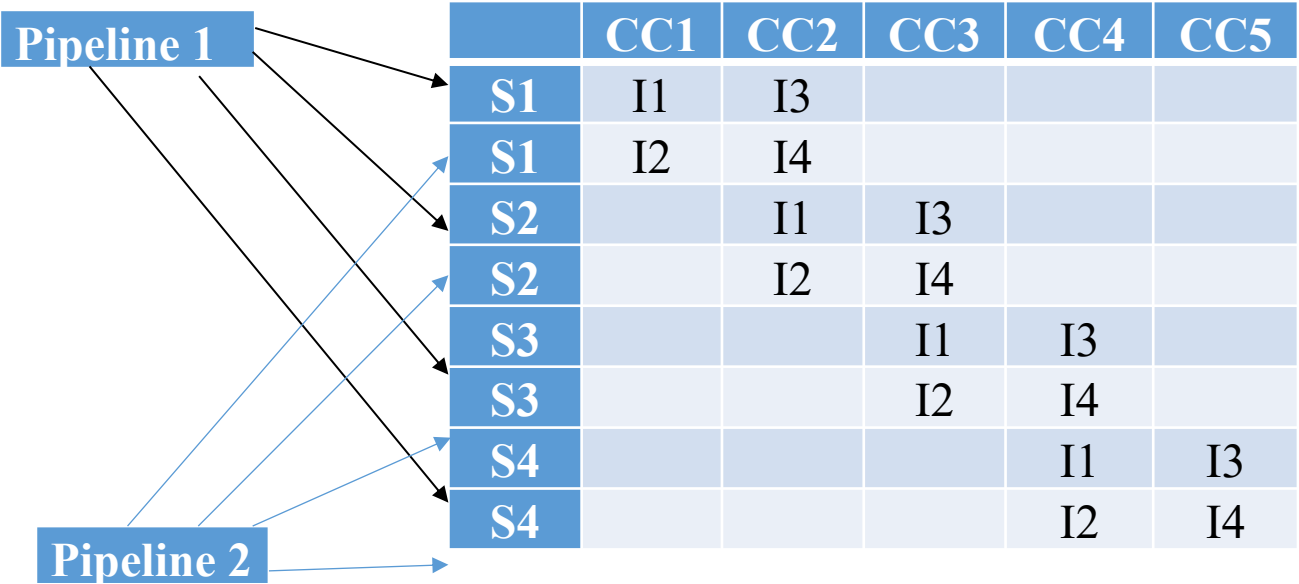
m number of instructions can be issued per clock cycle i.e. **K** number of clock cycles are required to complete the execution of the first m number of instructions out of N number of instructions. So the rest of the instructions whose executions are not over in K number of clock cycles is **N-m**

$\frac{1}{m}$ is the number of clock cycles to execute one instruction (clock per instruction (CPI))

So $\frac{N-m}{m}$ is the number of clock cycles to complete the execution of $N-m$ number of instructions

Space time diagram of a superscalar processor of degree 2 ($m=2$), $K=4$, $N=4$:

Four segments of Pipeline 1 and four segments of Pipeline 2 are shown by arrow in the following diagram.



Explanation:

All the instructions are independent

I1 and I2 enter into segment S1 of the two pipelines (as $m=2$) in CC1

I3 and I4 enter into segment S1 of the two pipelines whereas I1 and I2 enter into the segment S2 of the two pipelines in CC2

I3 and I4 enter into segment S2 of the two pipelines whereas I1 and I2 enter into the segment S3 of the two pipelines in CC3

I3 and I4 enter into segment S3 of the two pipelines whereas I1 and I2 enter into the segment S4 of the two pipelines in CC4, so the execution of I1 and I2 is over in CC4 and hence 4 clock cycles are required to complete the execution of the first two instructions (I1, I2) out of 4 instructions (I1, I2, I3, I4). The execution of 2 ($N-m=4-2=2$) instructions (I3 and I4) is not over in the first 4 clock cycles.

I3 and I4 enter into segment S4 of the two pipelines in CC5. So 1 more clock cycle is required to complete the execution of the rest of the two instructions i.e. I3 and I4. $\frac{N-m}{m} = \frac{4-2}{2} = 1$

Superpipeline processor

Exploits the fact that many pipeline segments execute instructions in less than half a clock cycle

In superpipeline processor of degree n , each clock cycle is divided into n number of short cycles and each short cycle is used to execute 1 instruction. So in this case clock per instruction (CPI) is $\frac{1}{n}$

Now consider a superpipeline processor of degree n ($n>1$) that is each segment of the pipeline in this processor executes operation (e.g. fetch instruction, decode instruction, fetch operand, execute instruction and store result) over each instruction in $\frac{1}{n}$ clock cycle. K is the number of segments in the pipeline and N is the number of instructions to be executed. So $K + \frac{N-1}{n}$ is the required number of clock cycles to complete the execution of N instructions by this processor.

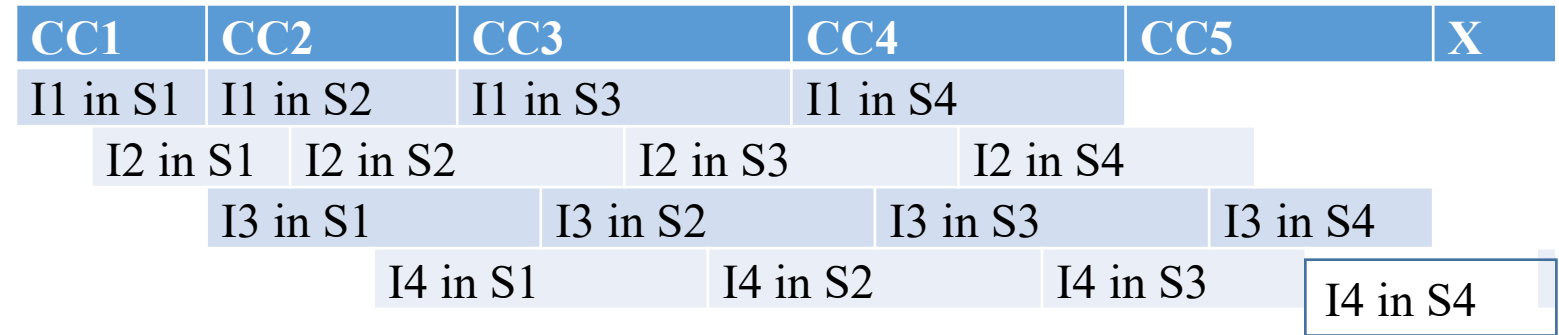
Explanation of $K + \frac{N-1}{n}$:

K number of clock cycles are required to complete the execution of the first instruction (I1). So the rest of the instructions whose executions are not over in **K** number of clock cycles is **N-1**

$\frac{1}{n}$ is the number of clock cycles to execute one instruction (clock per instruction (CPI))

So $\frac{N-1}{n}$ is the number of clock cycles to complete the execution of **N-1** number of instructions

X = $\frac{1}{2}$ of CC6 in the above figure



Explanation: All the instructions are independent

I1 enters into the segment S1 of the pipeline at the beginning of CC1 and I2 enters into the segment S1 of the same pipeline in half of CC1 (as first half of CC1 is sufficient to complete the operation over I1 by S1, so S1 becomes busy with I2 from second half of CC1)

I3 enters into the segment S1 of the same pipeline at the beginning of CC2 and I4 enters into the segment S1 of the same pipeline in second half of CC2

Now I1 enters into S4 in CC4. So the execution of I4 is over in CC4 and hence **4** (i.e. **K**) clock cycles are required to complete the execution of the first instruction (I1) out of 4 instructions (I1, I2, I3, I4). So **N-1** is $4-1=3$

Execution of I2 is over in half of CC5, execution of I3 is over in another half of CC5, execution of I4 is over in half of CC6 (shown as X in the above figure). So $\frac{1}{2} + \frac{1}{2} + \frac{1}{2} = 3 * \frac{1}{2} = (N-1) * \frac{1}{n}$

Problem

1. Pipeline has two stages, one for multiplication and one for addition. Each stage takes 10 ns. Then how much time is required to complete the execution of the following code segment?

for i = 1 to 100 do A[i]=B[i]*C[i]+D[i]

Ans. $K=2, N=100, (K+N-1)*10 \text{ ns} = (2+100-1)*10 \text{ ns}=1010 \text{ ns}$

2. A pipeline has 4 segments with time delays 60 ns, 50 ns, 90 ns, 80 ns. The interface latch has a delay of 10 ns. Find the clock frequency of the pipeline?

Ans. $\tau_m = \max \{60, 50, 90, 80\} = 90 \text{ ns}, d=10 \text{ ns}, \tau = \tau_m + d = 90+10=100 \text{ ns}$

So clock time (τ is 100 ns and clock frequency, $f=\frac{1}{\tau}$) = $\frac{1}{100} * 10^9 \text{ GHz}$

3. A 4 segment pipeline has segment delays as 150, 120, 160, 140 ns. The latch delay is 5 ns. Find the total time which is needed to process 1000 data items on this pipeline.

Ans. The time which is required to process the first data item using this pipeline is $150+5+120+5+160+5+140 = 585 \text{ ns}$

Average time to process each of the rest of the items is $\max\{150, 120, 160, 140\}+5 \text{ ns} = 165 \text{ ns}$

So the total required time is $585 + (1000-1)*165 \text{ ns}$

4. A pipeline is having speedup factor 10 and operating with an efficiency of 80%. What will be the number of stages in the pipeline?

Ans. $E_K = S_K / K$. $0.8 = 10 / K$, $K = 12.5 \cong 13$

5. A non-pipeline system takes 50 ns to process a task. The same task can be processed in a 6 segment pipeline with a clock cycle time 10 ns. Find the speedup ratio of the pipeline for 100 tasks.

Ans. The non-pipeline system takes $50 * 100$ ns to process 100 number of tasks.

In the pipeline system each segment takes 1 clock cycle to process a task. $K=6$, $N=100$, clock cycle time= 10 ns. So $6+(100-1)$ is the number of clock cycles to process 100 tasks using a 6 segment pipeline and the pipeline system takes $(6+(100-1)) * 10$ ns to complete the processing of 100 tasks. Hence

$$\text{Speedup} = \frac{100 * 50}{(6 + (100 - 1)) * 10} = 4.76$$